



Proyecto 1

Creación de un Chat de Texto TCP

Sánchez Girón Yael Adrian¹

¹Facultad de Ciencias, UNAM

Resumen

Palabras clave: Lorem, Ipsum, eusimod, sedes

1. Introducción

Aquí vendrá la introducción que pondré al final isdajfkaklfadklf

2. Modelado y Programación

Para iniciar con la realización de este chat, primero se tuvo que realizar una ardua investigación sobre las herramientas de forma detallada. Además, desde un inicio se nos notificó que estaba estrictamente prohibido utilizar el lenguaje de programación *Java*, lo cual resultó un pequeño impedimento ya que era la única herramienta sobre la que se poseía un dominio. Por ello, se inició la búsqueda de alternativas tanto para el cliente como para el servidor.

Después de investigar y evaluar varias opciones, se decidió utilizar el lenguaje de programación **Rust** para el servidor y el toolkit **Flutter** (que utiliza el lenguaje *Dart*) para el cliente y la interfaz gráfica. Se optó por Rust debido a su forma peculiar de manejar la memoria, conocida como *Rust Ownership*: Rust asigna cada dato en el heap a una variable propietaria única; cuando esa variable sale de su ámbito (scope), Rust libera automáticamente la memoria asociada, previniendo fugas de memoria y punteros colgados sin necesidad de gestión manual. Esto permite que Rust no dependa de un recolector de basura (Garbage Collector) como ocurre en Java [?].

Por su parte, Flutter es un framework *open-source* desarrollado por Google en 2015 con el objetivo de construir interfaces gráficas y compilarlas de forma nativa en múltiples plataformas. Su paradigma es Orientado a Objetos (OOP) y la sintaxis que maneja es muy similar a la de Java, por lo que

resultó ideal para el desarrollo del cliente. Al ser multiplataforma, se optó por compilar únicamente para Linux, macOS y Windows, ya que este chat será un programa exclusivo para entornos de escritorio. La configuración no fue un problema, pues el propio constructor de Flutter brinda las herramientas necesarias para estos sistemas operativos.

Una vez definidos los sistemas operativos, se continuó con el diseño estructural del proyecto. Se investigaron varios patrones de arquitectura de software, destacando el patrón Modelo-Vista-Controlador (MVC), la arquitectura Cliente-Servidor y el diseño Top-Down. Inicialmente, se optó por trabajar con el patrón MVC y una arquitectura Cliente-Servidor. Sin embargo, a medida que el proyecto avanzaba y se implementaban patrones de diseño de *conurrencia*, surgieron problemas de legibilidad; llegó un momento en el que era muy difícil identificar qué parte del código representaba cada función. Por esto, se implementó una refactorización completa de los archivos, tanto en el servidor como en el cliente.

Teniendo esto en cuenta, procederemos a explicar el servidor de Rust.

2.1. Servidor en Rust

Consideramos que el servidor es probablemente una de las partes menos triviales de implementar del proyecto. Esto se debe a que requiere la ejecución de acciones de forma asíncrona; es decir, el servidor debe ser capaz de realizar varias tareas al mismo tiempo de manera que las acciones bloqueantes puedan pausarse y permitir que el sistema procese otras peticiones.

La organización final de los archivos se basó en el patrón de diseño "Top-Down". Se partió de una

carpeta general llamada *rust_server* generada con la ayuda de la herramienta Cargo. Dentro de esta estructura, la carpeta *src* actúa como la raíz de todo el servidor. En su interior se crearon dos subcarpetas: *salas* y *usuarios*, las cuales adoptaron todos los *enums* creados en el archivo de *protocolo*. En estas subcarpetas se agregaron los algoritmos que son utilizados por el *manejador*. Esto se puede ver de forma más clara en el siguiente diagrama de componentes.

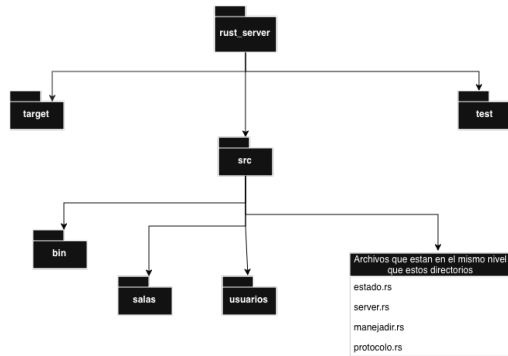


Figura 1: Diagrama de estructura del servidor en Rust (se omiten algunos archivos auxiliares para mayor claridad).

En la parte del diseño se detallará de forma más explícita el funcionamiento del servidor, pero en resumen, este debe cumplir con los siguientes puntos:

1. Establecer conexiones de tal manera que se puedan mantener activas por un tiempo indeterminado.
2. Recibir peticiones del cliente y procesarlas estrictamente de acuerdo con el protocolo establecido.
3. Almacenar los usuarios y el estado de las acciones dictadas por el protocolo.
4. Transmitir las acciones y mensajes necesarios al resto de los clientes conectados.
5. Estar programado de tal forma que sea tolerante a fallos (prácticamente indestructible ante desconexiones abruptas).
6. Realizar todas las acciones de red y lectura/escritura de forma asíncrona y concurrente.

El protocolo mencionado nos fue proporcionado desde el inicio como una guía y estándar, garantizando que cualquier cliente de la clase pudiera conectarse a cualquier servidor. Cabe destacar que el servidor representa la mitad de la arquitectura "Cliente-Servidor", un modelo donde múltiples clientes solicitan servicios o recursos de un nodo centralizado [?].

2.2. Cliente en Flutter

Para analizar al cliente, debemos retomar el patrón de diseño Modelo-Vista-Controlador (MVC). De acuerdo con el glosario de MDN Web Docs, este patrón se define de la siguiente manera [?]:

MVC (Modelo-Vista-Controlador) es un patrón en el diseño de software comúnmente utilizado para implementar interfaces de usuario, datos y lógica de control. Enfatiza una separación entre la lógica de negocios y su visualización. Esta "separación de preocupaciones" proporciona una mejor división del trabajo y una mejora de mantenimiento.

Las tres partes del patrón de diseño de software MVC se pueden describir de la siguiente manera:

1. **Modelo:** Maneja datos y lógica de negocios.
2. **Vista:** Se encarga del diseño y presentación.
3. **Controlador:** Enruta comandos a los modelos y vistas.

Bajo esta filosofía, el cliente desarrollado debe cumplir con las siguientes responsabilidades:

1. Conectarse al servidor de forma exitosa a través de un socket TCP.
2. Empaquetar y enviar los comandos ingresados de forma que coincidan exactamente con lo esperado por el servidor.
3. Traducir y deserializar los mensajes recibidos desde el servidor para actualizar la interfaz.
4. Proporcionar una Interfaz de Usuario (UI) intuitiva, aislando al usuario final de la complejidad de la red.

El objetivo principal de la interfaz gráfica es que el usuario final pueda comunicarse utilizando "lenguaje natural", mientras el cliente se encarga de traducir estas interacciones en los comandos estructurados (en formato JSON) necesarios para la comunicación con el servidor. De esta manera, el usuario interactúa cómodamente sin tener que preocuparse por tareas técnicas como escribir JSONs a mano o leer logs en una terminal.

2.3. Diseño general

Para poder entender de mejor manera el como funciona el servidor, se realizo el siguiente diagrama de secuencia:

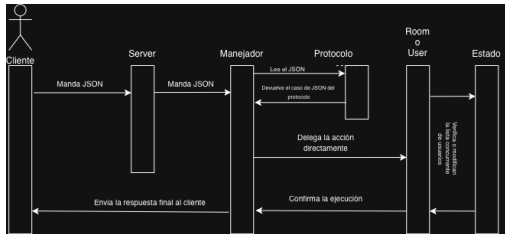


Figura 2: Diagrama de Secuencia del servidor en Rust.

Lo que muestra este diagrama es:

1. **Traducción:** *manejador* → *Protocolo* (pide leer el JSON) y luego *Protocolo* → *manejador* (devuelve el dato estructurado).
2. **Enrutamiento:** *manejador* → *Room o User* (delega la acción directamente, sin involucrar al protocolo).
3. **Memoria:** *Room o User* ↔ *Estado* (verifican o modifican la lista concurrente de usuarios).
4. **Retorno:** *Room o User* → *manejador* (confirman la ejecución).
5. **Salida:** *manejador* → *Cliente* (envía la respuesta final a través de la red).

Con esto nos damos una idea más clara de como es que el servidor debe de funcionar para que se cumpla con los requerimientos que fueron especificados.

Ahora, para poder entender como es que funciona el Cliente, se utilizará un diagrama del patron Modelo-Vista-Controlador

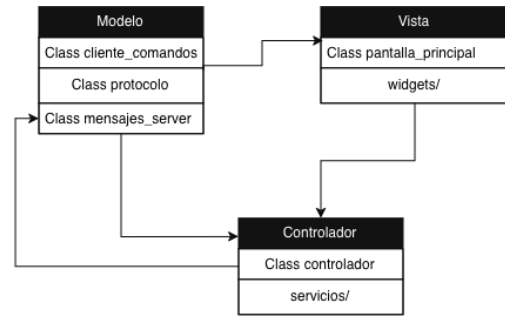


Figura 3: Diagrama de MVC del cliente en Faltter.

El diagrama de clases y paquetes (Figura 3) ilustra la implementación concreta del patrón Modelo-Vista-Controlador en el cliente de Flutter. La estructura garantiza una separación de responsabilidades, dividiendo el sistema en tres capas principales:

1. **Vista:** Compuesta por la clase `pantalla_principal` y la subcarpeta de `widgets/`. Esta capa es puramente visual y declarativa. Se encarga de renderizar la interfaz de usuario y capturar los eventos físicos (como el ingreso de texto o el clic en botones). Como indica el flujo, la Vista no toma decisiones lógicas; simplemente delega las acciones del usuario enviándolas hacia el Controlador.
2. **Controlador:** Actúa como el cerebro y mediador de la aplicación. Agrupa la clase `controlador` y la subcarpeta de `servicios/` (donde reside el manejo del socket TCP). Su función es recibir las interacciones de la Vista, ejecutar la lógica de red para comunicarse con el servidor en Rust y, una vez que obtiene una respuesta, procesar esos datos para enviarlos al Modelo.
3. **Modelo:** Contiene las clases `cliente_comandos`, `protocolo` y `mensajes_server`. Esta capa define las estructuras de datos estrictamente tipadas y representa el estado interno del cliente. Aquí se almacena la información estructurada que coincide con el protocolo del servidor (historial de mensajes, lista de usuarios, etc.).

El ciclo de vida de un evento comienza en la Vista, fluye hacia el Controlador para su procesamiento (envío y recepción por red) y desciende al Modelo para actualizar el estado de los datos.

Finalmente, el ciclo se cierra cuando el cambio de estado en el Modelo notifica a la Vista, provocando que la interfaz gráfica se reconstruya para mostrar al usuario la información más reciente.

Dentro del proyecto en la carpeta de `cliente_flutter` es posible observar la implementación de este patrón de diseño, sin embargo también se utilizó un poco del patrón de "Top-Down", específicamente en la carpeta de vista, ya que por las mismas razones que el servidor, llegó un punto en el cual el código era ilegible, y era muy difícil encontrar posibles bugs o errores de la lógica de la interfaz gráfica, es por eso que creé la subcarpeta de `widgets/`, en la cual engloba los puntos más primordiales para la interfaz gráfica, haciendo que el archivo de `pantalla_principal` sea mucho más legible.